

Algorithms

Chapter 7 Quicksort

Associate Professor: Ching-Chi Lin

林清池 副教授

chingchi.lin@gmail.com

Department of Computer Science and Engineering
National Taiwan Ocean University

Outline

- ▶ **Description of Quicksort**
- ▶ Performance of Quicksort 分析效能
- ▶ A Randomized Version of Quicksort 隨機的演算法
- ▶ Analysis of Quicksort

Quicksort

- ▶ Worst-case running time: $\Theta(n^2)$. 最差的情形為 $\Theta(n^2)$
- ▶ Best practical choice: 在實作上最好的選擇
 - ▶ Expected running time: $\Theta(n \lg n)$. 平均時間為 $\Theta(n \lg n)$
 - ▶ Constants hidden in $\Theta(n \lg n)$ are small. $\Theta(n \lg n) = C \cdot n \lg n$ 且 C 很微小
- ▶ Sorts in place.
與 heap sort 一樣是 in place sorting

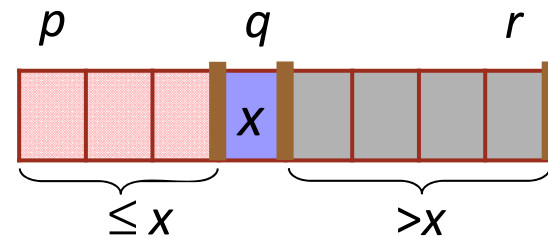
※ 雖然 heap sort 最差情形為 $\Theta(n \lg n) = C \cdot n \lg n$
但 C 很大, 所以在實作上仍選擇 quick sort

Description of quicksort

- ▶ Quicksort is based on the three-step process of divide-and-conquer.

用 divide and conquer 解決排序

- ▶ To sort the subarray $A[p\dots r]$:



- ▶ **Divide:** Partition $A[p\dots r]$, into two (possibly empty) subarrays $A[p\dots q-1]$ and $A[q+1\dots r]$, such that each element in the first subarray $A[p\dots q-1]$ is $\leq A[q]$ and $A[q]$ is $<$ each element in the second subarray $A[q+1\dots r]$.
- ▶ **Conquer:** Sort the two subarrays by recursive calls to quicksort.
- ▶ **Combine:** No work is needed to combine the subarrays, because they are sorted in place.

不需要 combine 的動作 → 不花任何的時間

The Quicksort procedure

QUICKSORT(A, p, r)

1. **if** $p < r$ 若左 < 右
2. **then** $q \leftarrow \text{PARTITION}(A, p, r)$ 作 divide
3. QUICKSORT($A, p, q-1$) 排左邊 subarray
4. QUICKSORT($A, q+1, r$) 排右邊 subarray

- ▶ Initial call is QUICKSORT($A, 1, n$).
- ▶ Perform the divide step by a procedure PARTITION, which returns the index q .

最花時間的是作 divide 的動作

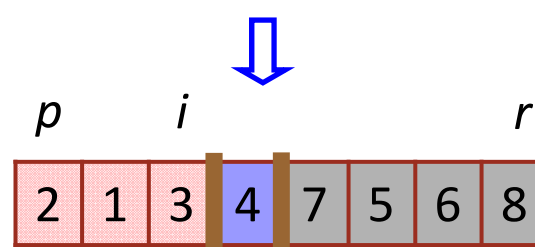
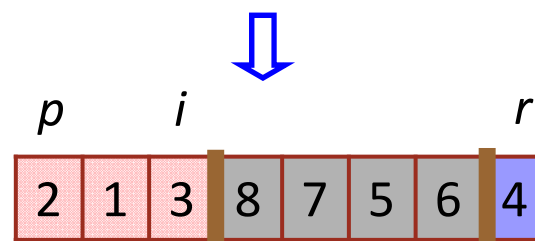
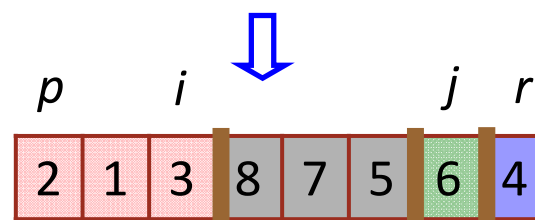
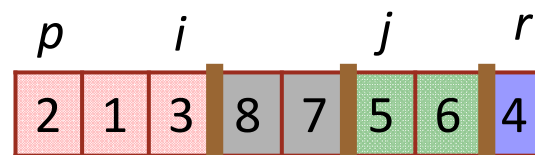
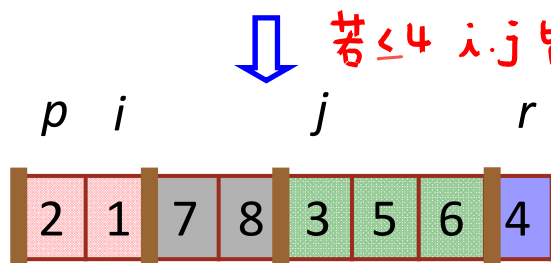
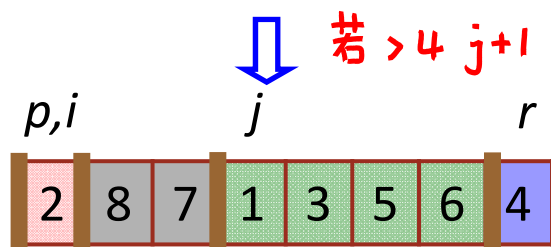
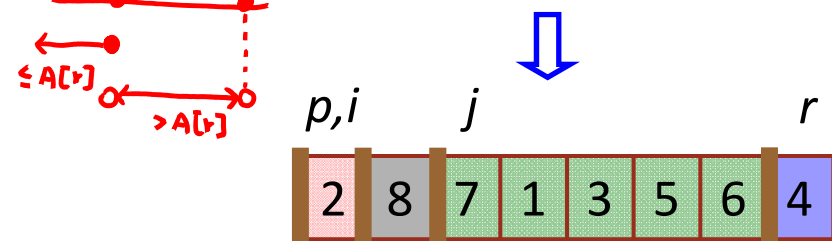
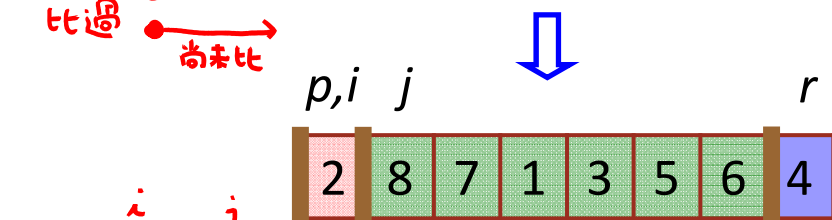
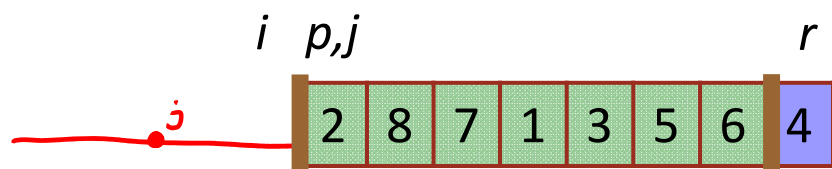
Partitioning the array

- ▶ Partition subarray $A[p\dots r]$ by the following procedure:

PARTITION(A, p, r)

1. $x \leftarrow A[r]$
 2. $i \leftarrow p - 1$
 3. **for** $j \leftarrow p$ **to** $r - 1$
 4. **if** $A[j] \leq x$
 5. $i \leftarrow i + 1$
 6. exchange $A[i] \leftrightarrow A[j]$
 7. exchange $A[i+1] \leftrightarrow A[r]$
 8. **return** $i + 1$
- Annotations:
- Lines 1 and 2: $\Theta(1)$ 以最後一個作比較基準
 - Lines 4, 5, and 6: $(n-1) \cdot \Theta(1)$ 和 x 作比較
 - Lines 7 and 8: $\Theta(1)$ 將比較基準 $A[r]$ 和 $A[i+1]$ 互換並回傳 $i+1$

- ▶ PARTITION always selects the last element $A[r]$ in the subarray $A[p\dots r]$ as the pivot. 永遠選擇最後一個當比較基準
- ▶ Time: $\Theta(n)$.



: pivot.
 : $\leq$ pivot.
 : $>$ pivot.
 : not examined.

Outline

- ▶ Description of Quicksort
- ▶ **Performance of Quicksort**
- ▶ A Randomized Version of Quicksort
- ▶ Analysis of Quicksort

Performance of quicksort

- ▶ The running time of quicksort depends on the partitioning of the subarrays: 時間複雜度和是否平均分割有關
- ▶ If the subarrays are balanced, then quicksort can run asymptotically as fast as mergesort. 分割平均則和 merge sort 一樣快
- ▶ If they are unbalanced, then quicksort can run asymptotically as slowly as insertion sort. 分割不平均則和 insertion sort 一樣差

$$T(n) = \underbrace{T(q)}_{\text{排左邊}} + \underbrace{T(n-q-1)}_{\text{排右邊}} + \underbrace{\theta(n)}_{\text{divide 的時間}}$$

Performance of quicksort

▶ **Worst-case partitioning:** 分割不公平

- ▶ Have 0 elements in one subarray and $n-1$ elements in the other subarray. - 一边 0 个 - 边 $n-1$ 个
- ▶ The recurrence is
$$\begin{aligned} T(n) &= T(n-1) + T(0) + \Theta(n) \\ &= T(n-1) + \Theta(n) = \Theta(1) + \Theta(2) + \dots + \Theta(n) \\ &= \Theta(n^2). \end{aligned}$$
- ▶ Occurs when the input array is sorted.

▶ **Best-case partitioning:** 分割平均, 刚好一半

- ▶ Occurs when the subarrays are completely balanced every time.
- ▶ Each subarray has $\leq n/2$ elements. 用 master method 的 case (II)
- ▶ The recurrence is
$$\begin{aligned} T(n) &\leq 2T(n/2) + \Theta(n) \\ &= \Theta(n \lg n). \end{aligned}$$

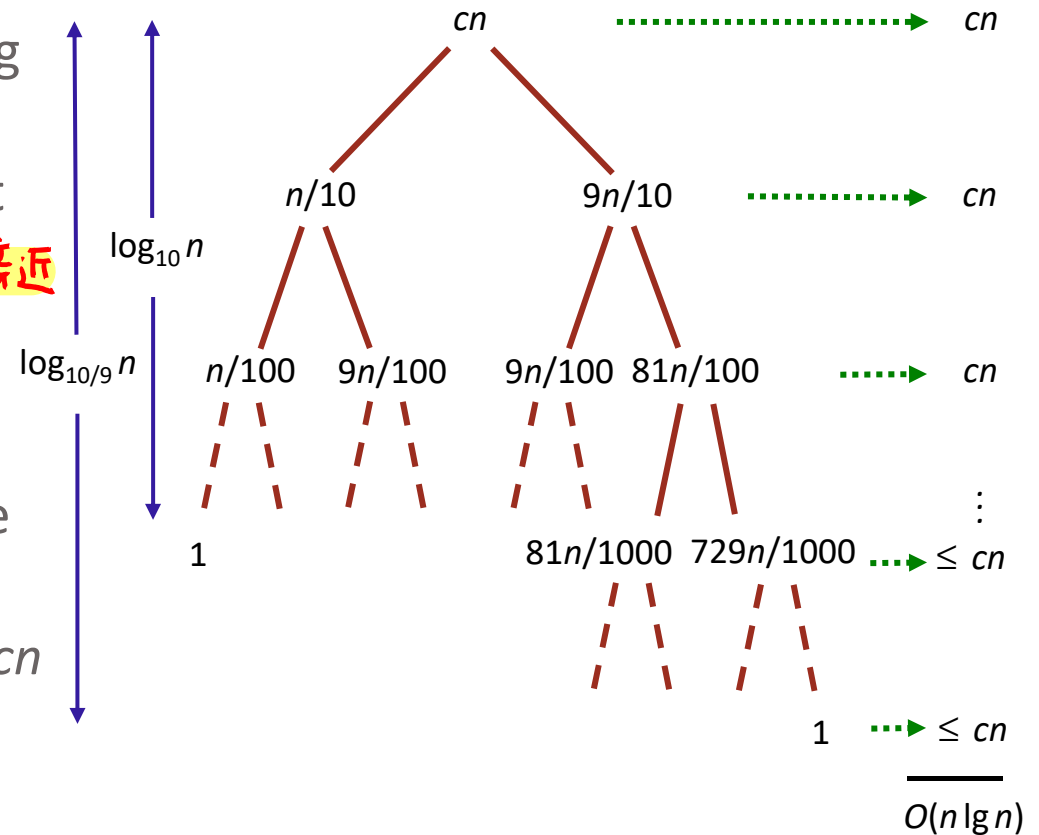
Balanced partitioning_{1/2}

► Balanced partitioning

- Quicksort's average running time is much closer to the best case than to the worst case. 平均時間和最佳時間接近

- Imagine that PARTITION always produces a 9-to-1 split, then the running time is 假設左:右 = 9:1

$$T(n) \leq T(9n/10) + T(n/10) + cn \\ = \Theta(n \lg n).$$



用遞迴樹產生答案, 用置換法驗證

Balanced partitioning_{2/2}

► **Intuition:** look at the recursion tree.

► It's like the one for $T(n) = T(n/3) + T(2n/3) + O(n)$ in Section 4.2.

► Except that here the constants are different; we get $\log_{10} n$ full levels and $\log_{10/9} n$ levels that are nonempty.

最高深度 ←

→ 最低深度

► As long as it's a constant, the base of the log doesn't matter in asymptotic notation.

► Any split of **constant proportionality** will yield a recursion tree of depth $\Theta(\lg n)$.

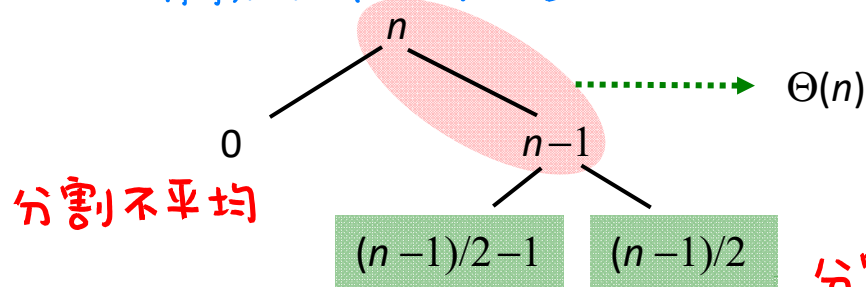
所以只要底數是常數, 即左右比是常數比, 時間就是 $\Theta(n \lg n)$

$$h = \log_{\frac{10}{9}} n = \frac{\log_2 n}{\log_2 \frac{10}{9}} = C \cdot \lg n = \Theta(\lg n)$$

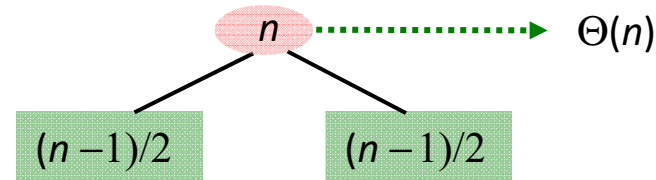
Intuition for the average case

- ▶ There will usually be a mix of good and bad splits throughout the recursion tree.
- ▶ Assume that levels alternate between best-case and worst-case splits. 假設好壞交錯發生

有時分割很平均、有時分割不平均



一直分割很平均



- ▶ The extra level in the left-hand figure only adds to the constant hidden in the Θ -notation. 左圖時間是右圖兩倍
 - ▶ Only twice as much work was done to get to that point.
 - ▶ Both figures result in $O(n \lg n)$ time.

Outline

- ▶ Description of Quicksort
- ▶ Performance of Quicksort
- ▶ **A Randomized Version of Quicksort** 隨機版本的 quick sort
- ▶ Analysis of Quicksort

Randomized version of quicksort_{1/2}

- ▶ In exploring the average-case behavior of quicksort, we have assumed that all input permutations are equally likely.
- ▶ This is not always true. 算平均時間時, 假設所有排序發生機率相同
- ▶ We use random sampling, or picking one element at random.
- ▶ Don't always use $A[r]$ as the pivot.
從 p 到 r 隨機選一個作基準, 不是永遠用 $A[r]$

RANDOMIZED-PARTITION(A, p, r)

1. $i \leftarrow \text{RANDOM}(p, r)$ 從 p 到 r 隨機選一個作基準
2. exchange $A[r] \leftrightarrow A[i]$ $A[i]$ $A[r]$ 交換
3. **return** PARTITION(A, p, r) 進行原本的 partition

Randomized version of quicksort_{2/2}

- ▶ Randomly selecting the pivot element will, on average, cause the split of the input array to be reasonably well balanced.

1. RANDOMIZED-QUICKSORT(A, p, r)
 2. if $p < r$
 3. then $q \leftarrow \text{RANDOMIZED-PARTITION}(A, p, r)$
 4. RANDOMIZED-QUICKSORT($A, p, q-1$)
 5. RANDOMIZED-QUICKSORT($A, q+1, r$)
- 隨機選一個作基準，平均而言
會將 input 分的平均

Outline

- ▶ Description of Quicksort
- ▶ Performance of Quicksort
- ▶ A Randomized Version of Quicksort
- ▶ **Analysis of Quicksort**

Analysis of quicksort

- ▶ We will analyze
 - ▶ the worst-case running time of QUICKSORT and RANDOMIZED-QUICKSORT (the same), and
 - ▶ the expected (average-case) running time of RANDOMIZED-QUICKSORT.

- ▶ **Worst-case analysis:** $T(n) = O(n^2)$.

- ▶ Recurrence for the worst-case:

$$T(n) = \max_{0 \leq q \leq n-1} (T(q) + T(n - q - 1)) + \Theta(n).$$

有n種方法,取最差

- ▶ Because PARTITION produces two subproblems, totaling size $n - 1$, q ranges from 0 to $n - 1$.

Worst-case analysis

► **Guess:** $T(n) \leq cn^2$, for some c . 猜答案

► Substituting our guess into the recurrence: 用置换法

$$\begin{aligned} T(n) &= \max_{0 \leq q \leq n-1} (T(q) + T(n-q-1)) + \Theta(n) \\ &\leq \max_{0 \leq q \leq n-1} (cq^2 + c(n-q-1)^2) + \Theta(n) \\ &= c \cdot \max_{0 \leq q \leq n-1} (q^2 + (n-q-1)^2) + \Theta(n) \end{aligned}$$

► The maximum value of $(q^2 + (n-q-1)^2)$ occurs – when q is either 0 or $n-1$. (second derivative)

► This means that $\max_{0 \leq q \leq n-1} (q^2 + (n-q-1)^2) \leq (n-1)^2 = n^2 - 2n + 1$.

► Therefore, $T(n) \leq cn^2 - c(2n-1) + \Theta(n)$ 算出的结果

$$\leq cn^2 \text{ 目标}$$

$$= O(n^2)$$

choose c so that

$$c(2n-1) \geq \theta(n)$$

找得出 c 值 $\Rightarrow$ 得证

Average-case Analysis_{1/5} 平均的時間

- ▶ **Average-case analysis:** $T(n) = O(n \log n)$.
 - ▶ The dominant cost of the algorithm is partitioning.
 - ▶ PARTITION is called at most n times. 最關鍵的花費是 partition 的時間
 - ▶ The amount of work that each call to PARTITION does is a constant plus the number of comparisons that are performed in its **for loop**. partition 的時間 = $O(1)$ + for loop 中比較的時間
 - ▶ Let X = the total number of comparisons performed in all calls to PARTITION. 令 x 為所有 for loop 中比較的時間
 - ▶ Therefore, the **total work** is $O(n + X)$.
 $n \cdot (O(1) + \text{for loop 中比較的時間}) = O(n + x)$

任兩個 element 最多比較 - 次

(I) 只跟基準比

(II) 基準不再跟別人比

(III) 左右往後不會再相比

Average-case Analysis_{2/5}

- ▶ We will now compute a bound on the overall number of comparisons.
- ▶ For ease of analysis: 方便分析
 - ▶ Rename the elements of A as $z_1, z_2, \dots, z_n$, with z_i being the i th smallest element. z_i 是第 i 小的元素
 - ▶ Define the set $Z_{ij} = \{z_i, z_{i+1}, \dots, z_j\}$ to be the set of elements between z_i and z_j , inclusive.
- ▶ Each pair of elements is compared at most once:
 - ▶ Elements are compared only to the pivot element, and
 - ▶ The pivot element is never in any later call to PARTITION.
- ▶ The expectation of total number of comparisons performed by the algorithm is $E[X] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n \Pr\{z_i \text{ is compared to } z_j\}$. $\times$

Average-case Analysis_{3/5}

- ▶ Consider the input: 2, 8, 7, 1, 3, 5, 6, 4 and the pivot is 4,
 - ▶ None of the set {2, 1, 3} will ever be compared to any of the set {5, 6, 7, 8}. 比基準大者和比基準小者, 往後不會再相比
- ▶ Once a pivot x is chosen such that $z_i < x < z_j$, then z_i and z_j will never be compared at any later time.
 $z_i < x < z_j$ 且 x 比 z_i 和 z_j 早選中 $\Rightarrow z_i$ 和 z_j 不會互比
- ▶ If either z_i or z_j is chosen before any other element of Z_{ij} , then it will be compared to all the elements of Z_{ij} , except itself.
 若 z_i or z_j 比 Z_{ij} 中其他元素早選中, 則 z_i or z_j 將和 Z_{ij} 中所有元素相比

3 和 7 比的機率	$\Rightarrow$ 3 或 7 要比 4, 5 和 6 先選
$\Rightarrow$ 1 和 2 先選不會影響	$\Rightarrow$ 3 先選機率 $\frac{1}{7-3+1}$
3 和 7 仍然同時在 1 或 2 的右邊	$\Rightarrow$ 7 先選機率 $\frac{1}{7-3+1}$
$\Rightarrow$ 8 也不會同時在 8 的左邊	$\Rightarrow$ 7 和 3 比的機率 $\frac{2}{7-3+1}$
4, 5 和 6 會將 3 和 7 分開	

Average-case Analysis_{4/5}

► Therefore,

$$\begin{aligned}\Pr\{z_i \text{ is compared to } z_j\} &= \Pr\{z_i \text{ or } z_j \text{ is the first pivot chosen from } Z_{ij}\} \\ &= \Pr\{z_i \text{ is the first pivot chosen from } Z_{ij}\} \\ &\quad + \Pr\{z_j \text{ is the first pivot chosen from } Z_{ij}\}\end{aligned}$$

$$\begin{aligned}&= \frac{1}{j-i+1} + \frac{1}{j-i+1} \\ &= \frac{2}{j-i+1}.\end{aligned}$$

z_i 被選中的機率 z_j 被選中的機率

► Substituting into the equation for $E[X]$:

$$\text{► } E[X] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n \Pr\{z_i \text{ is compared to } z_j\} = \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j-i+1}.$$

期望值

Average-case Analysis_{5/5}

► Let $k = j - i$, then $E[X] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j-i+1}$ $k = j - i$

$$= \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} \frac{2}{k+1}$$
$$< \sum_{i=1}^{n-1} \sum_{k=1}^n \frac{2}{k}$$
$$= \sum_{i=1}^{n-1} O(\lg n)$$
$$= O(n \lg n).$$

$\sum \frac{2}{k}$
 $= 2 \sum \frac{1}{k}$
 $= 2 \left(\frac{1}{1} + \frac{1}{2} + \dots + \frac{1}{n} \right)$
 $= 2 (\lg n + O(1))$
 $= O(\lg n)$

- So the expected running time of quicksort, using RANDOMIZED-PARTITION, is $O(n \lg n)$.